Egne MCP-værktøjer

muligheder, opskrifter og anbefaling

Research 2/11 2026. Spørgsmålet var »kan vi lave vores egne værktøjer?«. Svaret er at vi **allerede har dem**: Autopilot-plugin'et registrerer WordPress-abilities, og AI Engine viser dem som MCP-værktøjer. Det der mangler er ikke en ny server, men flere rigtige værktøjer oven på den registrering vi har.

Metode: kildelæsning af `plugins/steamgears-autopilot/includes/abilities.php`, AI Engine 3.9.0 (`labs/mcp-abilities.php`, `labs/mcp.php`) og mcp-adapter 0.7.1 (`includes/Abilities/McpAbilityExposure.php`). Live-tallene kommer fra ../mcp-auth.md, hvor 206 værktøjer er målt med OAuth-token.

1. Udgangspunktet: vi har en værktøjsflade i forvejen

`SGAP_Abilities` registrerer sine abilities med kategori `steamgears-autopilot` og det meta der skal til for at begge MCP-servere viser dem:

```
'meta' => [
    'public'     => true,                                    // læses af servere der kun kigger på
højniveau-flaget
    'mcp'        => ['public' => true, 'type' => 'tool'],
    'annotations'=> ['readonly' => $readonly, 'destructive' => $destructive, 'idempotent' =>
$idempotent],
],
```

- **AI Engine** lægger et værktøj oven på hver ability der er offentlig. Navnet laves af `tool_name()`: alt ud over `a-zA-Z0-9_-` bliver `_`, og navne over 64 tegn får en hash-suffiks. Derfor hedder vores værktøjer `steamgears-autopilot_status`, `steamgears-autopilot_queue-post` osv. — den familie der er målt til 31 værktøjer på sitet.
- **Access level** udledes af annotationerne: `readonly` → `read`, ellers `write`/`admin`. Det er niveauet AI Engine håndhæver i `execute_tool()`, ikke en etiket.
- **mcp-adapter** kræver `meta.mcp.public = true` (eller `meta.public`), se `McpAbilityExposure::is_public()`. Vores meta opfylder begge, så de samme abilities kan ses gennem adapterens `discover-abilities`/`execute-ability`.
- **Kollisioner:** bruger to abilities samme værktøjsnavn, springer AI Engine den anden over og logger det én gang i døgnet. Navnet skal derfor være unikt på tværs af alle plugins.

Konsekvens: **én registrering, to flader.** Vi skal ikke vælge mellem AI Engine og adapteren for at få egne værktøjer — vi skal bare registrere abilityen ordentligt.

Målt lokalt: 32 abilities, hvoraf 10 læsende

Ved at køre plugin'et mod test-stubberne (samme harness som `plugins/tests/smoke.php`) og fyre `wp_abilities_api_init` af, registrerer 7.23.0 **32 abilities** — **10 læsende**, **22 skrivende** — og alle 32 har `meta.mcp.public = true`. Tællingen er gjort 2/11 2026 med `do_action('wp_abilities_api_init')` efterfulgt af `array_keys($GLOBALS['abilities'])`, og den er reproducerbar uden WordPress.

Den stemmer med sitet: main målte **31** `steamgears-autopilot_*`-værktøjer på 7.22.0, og forskellen til de 32 er netop `meta-audit`, som først findes fra 7.23.0. Det gør tallet til en kontrol af at en ny ability lægger præcis ét værktøj oveni — ikke nul og ikke to.

2. Fire måder at tilføje et værktøj

Vej	Hvor koden bor	Virker når	Styrke	Svaghed
A. Ability i Autopilot (anbefalet)	plugins/steamgears-autopilot/includes/	begge MCP-servere, alle klienter	samme kode som resten af plugin'et, versionsstyret i repoet, CI kan teste den	kræver en plugin-release og «Plugins → Update» på sitet
B. mwai_mcp_tools-filter i et mu-plugin	wordpress/mu-plugins/	kun AI Engine	hurtigt, ingen release, kan skrives som rent array	ikke en ability, så adapteren ser den ikke; usynlig for Abilities-API'et og for AI Engines egen værktøjsliste i wp-admin
C. Kun mcp-adapter	enhver plugin-fil	kun adapteren	lille overflade	adapteren har kun tre meta-værktøjer, så alt går gennem execute-ability; intet ekstra vindue
D. Egen stdio-server i repoet	tools/	kun lokale klienter	ingen site-adgang nødvendig, testbar i CI	kan ikke nås af eksterne klienter, og kan ikke læse sitet

A er valgt fordi den rammer begge flader og fordi vores abilities allerede er den vej Autopilot styrer køen og publiceringen. B er kun til noget der skal prøves af i en fart, og som ikke skal leve længe.

3. Opskrift A — sådan tilføjes et værktøj

1. **Vælg navnet** som anden del af ability-navnet: `steamgears-autopilot/<navn>`. Værktøjet hedder så `steamgears-autopilot_<navn>`. Tjek at det ikke findes.
2. **Skriv definitionen** i `definitions()` som `[$label, $description, $properties, $required, $execute, $readonly, $idempotent]`. Beskrivelsen er det klienten læser for at vælge værktøjet — skriv hvad den gør, hvad den returnerer, og hvornår den ikke skal bruges.
3. **Skriv callbacken** som statisk metode: tager det validerede input-array, returnerer et array eller `WP_Error`. Ingen `echo`, ingen `exit` — svaret skal kunne serialiseres.
4. **Sæt annotationerne rigtigt.** Læser den kun, er `$readonly = true`; ellers bliver værktøjet `write` og rammer Drafts Only-spærren på sitet. Det er ikke en fejl — det er spærren der gør sitet sikkert.
5. **Test i CI** med plugin'ets smoke-checks, og verificér mod sitet med `python3 tools/mcp_rest.py --call steamgears-autopilot_<navn> '{}'` når der er et token.

Eksempel på et læsende værktøj der mangler i dag (meta-revisitation, som i dag gøres manuelt):

```
'meta-audit' => [
    'Audit post meta',
    'Checks title and meta-description lengths for published posts: how many are missing a
description, '
    . 'how many exceed the limits, and which posts are worst. Read-only.',
    [ 'limit' => [ 'type' => 'integer', 'minimum' => 1, 'maximum' => 200 ] ],
    [], [__CLASS__, 'meta_audit'], true, true
],
```

4. Opskrift B — AI Engines filter, hvis det skal prøves af uden release

```
add_filter('mwai_mcp_tools', function (array $tools): array {
    $tools[] = [
        'name'        => 'steamgears_roadmap_gap',
        'description' => 'Which roadmap rows are still open for the current quarter.',
        'inputSchema' => ['type' => 'object', 'properties' => new stdClass()],
        'accessLevel' => 'read',
        'annotations' => ['readOnlyHint' => true],
    ];
    return $tools;
});
```

Filteret køres i `get_tools_list()` **før** AI Engine bygger sit access-level-kort, så `accessLevel` skal stå i arrayet — ellers behandles værktøjet som `admin`. Kaldet selv skal fanges med `mwai_mcp_callback`, og `mwai_mcp_needs_approval` kan kræve ejerens godkendelse pr. værktøj. Prisen er at værktøjet ikke findes i Abilities-API'et, så adapteren og AI Engines egen oversigt i wp-admin ikke viser det.

5. Hvad vi bør bygge først

Rangeret efter hvor meget manuelt arbejde det fjerner, og alle læsende, fordi skrivning er spærret af Drafts Only indtil ejeren slår det fra:

1. **meta-audit** — titel- og metabeskrivelseslængder på tværs af publicerede poster. I dag en manuel runde med 11 rettelser (meta-fixes-og-produktlinks-2026-11-02.md). **Bygget 2/11 2026 i plugin 7.23.0:** `includes/meta-audit.php` (`SGAP_Meta_Audit`), ability `steamgears-autopilot/meta-audit`, 11 nye smoke-checks (208 → 219). Læser Rank Math og derefter Yoast, tæller manglende/for lange/for korte beskrivelser og sorterer de værste poster først. Skriver intet — testen kontrollerer at der ikke kommer `INSERT`/`UPDATE` i `$wpdb`-loggen. **Ikke verificeret mod sitet endnu:** det kræver at 7.23.0-zip'en er merget til `main` og hentet via «Plugins → Update», plus et OAuth-token i det worktree der kalder `tools/mcp_rest.py`.
2. **redirect-audit** — hvilke mål i roadmap'et mangler en omdirigering, læst fra Redirection-plugin'et. Roadmap S0.4 står stadig på 0 % fordi gruppen ikke er oprettet.
3. **link-audit** — døde interne links og links til kladder. Dækker samme grund som quality-gaten, men kaldes ovenfra i stedet for ved kø-tid.
4. **schema-audit** — om product-schemaet faktisk ligger i JSON-LD på de ni guides. Verifikationen i produkt-schema-fix-2026-11-02.md er i dag manuel.

Først når Drafts Only slås fra, giver skrivende værktøjer mening — og de bør da have `destructive` -annotation og gerne approval, så et forkert kald ikke kan slette en post.

5b. Pro-sættene: hvad de er — og hvorfor vi ikke skal genskabe dem

Meow Apps' egen tutorial siger det direkte: *»The WordPress tools are free; Pro adds tool sets for plugins, themes, the database, WooCommerce, Polylang and WPML.«* Spørgsmålet er derfor ikke om vi skal kopiere »alle Pro-værktøjerne«, men om de seks sæt dækker noget vi ikke har.

Målt i den frie 3.9.0-kilde: `switch_theme` **0**, `install_plugin` **0**, `activate_plugin` **0**, `run_query` **0**, `wc_/woocommerce` **0**, `polylang/wpml` **0** — mens `wp_list_plugins`, `wp_get_option`, `wp_update_option`, `wp_edit_menu`, `wp_edit_widgets`, `wp_get_users`, `wp_create_user` og `wp_update_user` findes i den frie udgave.

Pro-sæt	Har vi dækning i dag?	Beslutning
Plugins (installér, aktivér, skift)	list-plugins, manage-plugin, update, enable-full-access i vores plugin; wp_list_plugins i den frie AI Engine	Vi bygger ikke installér/aktivér: MCP-fladen kan læse plugins men ikke installere dem, og et værktøj der skifter plugin på et produktionssite er en udrulningsknap uden fortrydelse

Tema (filer, skift)	wp_get_templates, wp_get_site_style, wp_update_site_style, wp_edit_blocks	Fildredigering nej — det er PHP i en aktiv template. Styling er dækket
Database (rå SQL)	wp_get_option/wp_update_option i den frie AI Engine	Nej. En rå query kan ikke annoteres meningsfuldt som læsende eller destruktiv, og den kan ikke fortrydes
WooCommerce	Målt i Woo 11.2.1: syv domæne-abilities, alle mcp.public = true, registreret ubetinget — plus ni udfasede REST-spejle med show_in_rest	Dækket af Woo gennem AI Engines bro, uden Pro (se §5c). Vi bygger ikke CRUD — men en læsende product-audit ville give mening
Polylang / WPML	Sitet er ettsproget	Ikke relevant

Konklusion: vi skal ikke genskabe Pro-værktøjerne. Plugin- og temadelen er enten dækket eller bevidst udeladt, WooCommerce dækkes af Woo selv gennem Abilities API, database- og temasættene er den slags værktøjer man fortryder, og flersproglighed er ikke en problemstilling her. Det vi mangler er ikke **flere knapper**, men **domskraft**: læsende audits der siger hvad der er galt, så de manuelle runder forsvinder. Rækkefølgen i §5 står derfor uændret, og `product-audit` (varianter, priser, manglende billeder, schema) bør komme ind som nummer fem.

5c. Woo-adgangen er målt, ikke antaget

Spørgsmålet var om produkt-CRUD reelt er tilgængeligt gennem AI Engine uden Pro. Fem trin, alle afgjort i koden i stedet for i dokumentationen:

1. **Registrering:** `src/Internal/Abilities/AbilitiesRegistry.php:28` kalder `AbilitiesLoader::init()`, som hænger `register_abilities` på `wp_abilities_api_init`/`abilities_api_init` — ingen feature-flag, ingen licens, kun `function_exists( 'wp_register_ability' )`.
2. **Publicering:** alle syv sætter `'meta' => [ 'mcp' => [ 'public' => true ] ]` (`src/Internal/Abilities/Domain/*.php`, fx `ProductUpdate.php:55`).
3. **Broen:** AI Engines `labs/mcp-abilities.php:42-48` tager alt med `mcp.public === true` **eller** `show_in_rest`, og et eksplicit `mcp.public = false` vinder. Broen er ikke en indstilling — den hægtes på `mwai_mcp_tools` når `wp_get_abilities` findes.
4. **Rettigheder:** de følger ability'en, ikke broen. `ProductUpdate::can_update_product()` kræver `wc_rest_check_post_permissions( 'product', 'edit' )`, så MCP-brugeren skal kunne redigere produkter.
5. **Forudsætning:** WordPress med Abilities API. Sitet svarer `401 rest_forbidden` på `/wp-json/wp-abilities/v1/abilities` og `.../categories` — ruterne findes altså, de kræver bare login.

Svar: ja. Syv Woo-værktøjer (`woocommerce_product-create`, `-update`, `-delete`, `products-query`, `orders-query`, `order-update-status`, `order-add-note`) er tilgængelige gennem AI Engine uden Pro — plus op til ni udfasede REST-spejle, fordi de bærer `show_in_rest` og derfor også går gennem broen. Tilsammen op til **16 værktøjer**, som alle lander i `ability_bridge`-spanden når `--split` køres.

Det ændrer ikke beslutningen om ikke at bygge CRUD: funktionaliteten findes allerede, og Wos egne `permission_callback`s er bedre end noget vi ville skrive. Det ændrer til gengæld «hvorfor» vi ikke bygger den — ikke fordi Pro mangler, men fordi den er der, og dobbeltarbejde ville give to sandheder om samme produkt.

6. Sikkerhed og governance

- **Kun læsende værktøjer gøres offentlige først.** `meta.mcp.public` er ikke en etiket — det er hvad en fremmed MCP-klient kan kalde, når den først har et token.
- **permission_callback skal stå i hver ability.** Vores bruger `can_manage()`; en ability uden rettighedstjek er en åben dør i samme øjeblik nogen får et token.
- **Annotér destruktive værktøjer.** Uden `destructiveHint` kan en klient tro at en sletning er en pæn opdatering.

- **Drafts Only er backstop, ikke design.** Den stopper skrivning på AI Engine-fladen, men ikke nødvendigvis gennem adapterens `execute-ability` — så abilities skal selv tjekke rettigheder og tilstand.
- **Tool-navne er en delt navnerum.** To plugins der registrerer samme navn betyder at den ene forsvinder lydløst for klienten.

7. Det der stadig kræver ejeren

- **Drafts Only**: slås den fra, bliver de skrivende værktøjer brugbare — det er en beslutning om risikoprofil.
- **Hvilken flade der er kanonisk**: AI Engine har 206 værktøjer og er den målte vej ind; adapteren er et supplement med tre meta-værktøjer. Skal vi have egne værktøjer, er abilities den fælles mængde, og derfor det rigtige sted at bygge.
- **Måling mangler**: hvor mange af de 206 der kommer fra vores egne abilities kontra AI Engines indbyggede. Det kræver `tools/list` med et gyldigt token og er næste skridt, når OAuth er godkendt igen. Efter 7.23.0-udgivelsen er `steamgears-autopilot/meta-audit` det værktøj der beviser at en ny ability når hele vejen frem: `python3 tools/mcp_rest.py --call steamgears-autopilot_meta-audit '{"scan":5,"limit":5}'`.

Referencer

- ../mcp-auth.md — credential-matrix, OAuth-vejen og de 206 målte værktøjer
- mcp-vaerktoejer-2026-11-02.md — AI Engines 74 kerneværktøjer og adapterens tre
- mcp-adapter-abilities-2026-11-02.md — adapterens abilities og brugbarhedsscore
- `plugins/steamgears-autopilot/includes/abilities.php` — vores egen registrering
- AI Engine: <https://meowapps.com/ai-engine/> · Abilities API: <https://developer.wordpress.org/news/2025/11/introducing-the-wordpress-abilities-api/>